Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	M.Tech Computer Science & Engineering	Name	**Pandiarajan D**
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	**Sem V**

Experiment 3: Ensemble Classification Models (Decision Tree, Random Forest, AdaBoost, Gradient Boost, XGBoost)

Objective

To implement Decision Tree, Random Forest, AdaBoost, Gradient Boost, and XGBoost classifiers on the given dataset, evaluate their performance using Accuracy, Precision, Recall, F1-score, and K-Fold cross-validation, and analyze the results with Confusion Matrix and ROC Curve.

Libraries Used

- `pandas`, `numpy` for data handling
- `matplotlib`, `seaborn` for visualization
- `scikit-learn` for ML models and evaluation
- `xgboost` for XGBoost implementation

Theoretical Description of Models

Decision Tree

A Decision Tree is a non-parametric supervised learning algorithm that recursively partitions the feature space into simple regions. The objective is to learn a series of 'if-else' questions about the features that lead to the purest possible nodes (i.e., nodes containing samples predominantly from a single class).

The most common objective function for classification is to maximize the *Information Gain* at each split, which is equivalent to minimizing impurity (e.g., Gini Impurity or Entropy).

The Gini Impurity for a node t is defined as:

$$G(t) = 1 - \sum_{i=1}^{c} (p(i|t))^2$$

where $p(i|t)$ is the proportion of samples belonging to class i at node t.

AdaBoost (Adaptive Boosting)

AdaBoost is a boosting algorithm that combines multiple weak learners (typically shallow trees) into a strong classifier. It works sequentially, focusing on the mistakes of the previous learner.

The algorithm:

1. Initially assigns equal weights to all training samples.
2. Iteratively:
   (a) Trains a weak learner on the weighted dataset.
   (b) Increases the weights of the misclassified samples.
   (c) Calculates a weight for the weak learner based on its accuracy.

The final model is a weighted sum of all weak learners:

$$F(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(x)\right)$$

where α_m is the weight of the m-th weak learner $h_m(x)$.

Gradient Boosting

Gradient Boosting is another sequential ensemble technique that builds models additively. Unlike AdaBoost, which adjusts weights, Gradient Boosting fits each new weak learner to the *pseudo-residuals* (negative gradient) of the current model's loss function.

For a differentiable loss function $L(y, F(x))$ (e.g., log-loss), the algorithm:

1. Initializes a model with a constant value: $F_0(x) =_\gamma \sum_{i=1}^{n} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) Computes residuals: $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$.
   (b) Fits a weak learner $h_m(x)$ to the residuals.
   (c) Updates the model: $F_m(x) = F_{m-1}(x) + \nu \cdot \gamma_m h_m(x)$, where ν is the learning rate.

It greedily minimizes the loss function by moving in the direction of the steepest descent.

XGBoost (Extreme Gradient Boosting)

XGBoost is an advanced, highly optimized implementation of Gradient Boosting. Its key innovation is a more sophisticated objective function that includes a *regularization term* to control model complexity and prevent overfitting.

The objective function at the t-th iteration is:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

where the regularization term $\Omega(f)$ is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Here, T is the number of leaves, w_j is the score on leaf j, γ is the complexity penalty, and λ is the L2 regularization term. XGBoost uses second-order Taylor approximation to quickly optimize this objective.

Random Forest

Random Forest is an ensemble method that combines multiple decorrelated Decision Trees via *Bagging* (Bootstrap Aggregating). The algorithm introduces randomness in two ways:

1. Each tree is trained on a different bootstrap sample of the training data.
2. At each split, only a random subset of features is considered for partitioning.

The final prediction is made by majority voting (for classification) or averaging (for regression) across all trees. This process reduces variance and mitigates overfitting, leading to better generalization than a single Decision Tree.

Stacked Ensemble

Stacking combines multiple base models (heterogeneous or homogeneous) by using their predictions as new features to train a final *meta-learner*.

The algorithm:

1. Splits the training data into folds.
2. Trains each base model on the training folds and makes predictions on the validation fold (to avoid overfitting).
3. Concatenates these out-of-fold predictions to form a new feature matrix.
4. Trains the meta-learner (e.g., Logistic Regression) on this new feature matrix.

The final prediction is made by first getting predictions from all base models and then feeding them as input to the meta-learner.

Implementation Steps

1. Load and preprocess the dataset (handle missing values, normalization).
2. Perform Exploratory Data Analysis (EDA).
3. Split dataset into training and testing sets.
4. Train models: Decision Tree, Random Forest, AdaBoost, Gradient Boost, and XGBoost.

5. Evaluate using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC Curve.

6. Perform 5-Fold Cross Validation.

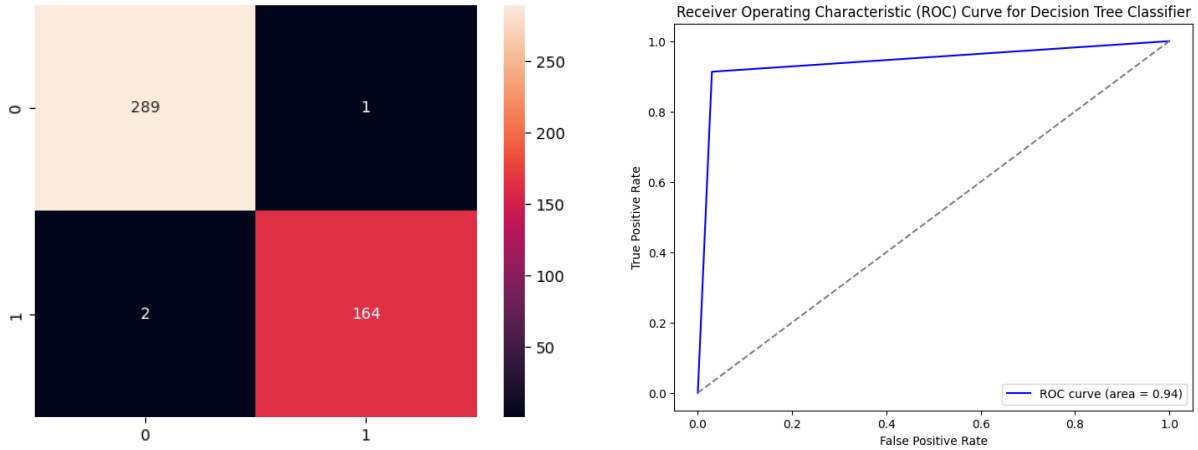
7. Compare results and record observations.

Decision Tree Classifier

Code Snippet

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

dt_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=None,
    random_state=42
)
dt_model.fit(X_train, y_train)

scores = cross_val_score(dt_model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

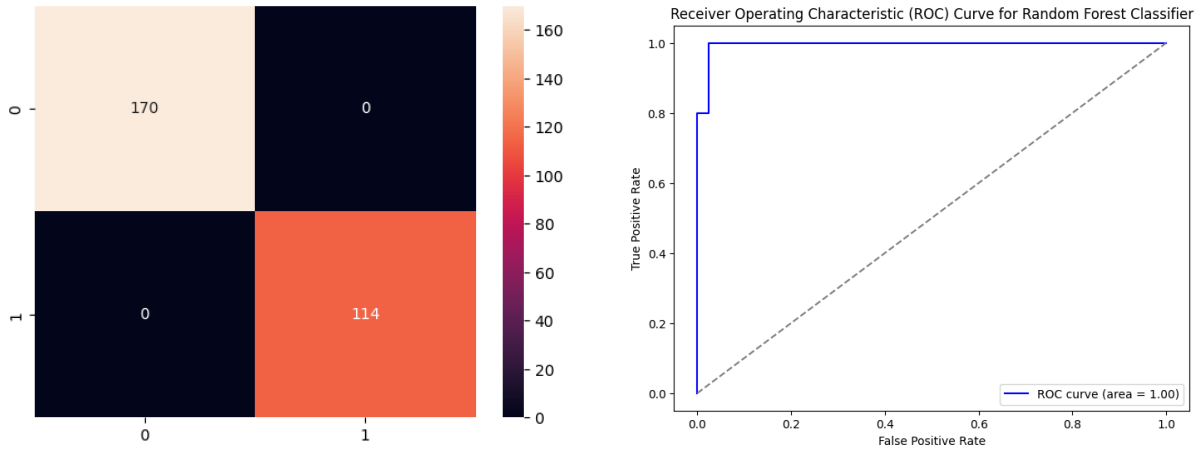
Figure 1: Decision Tree Evaluation

Random Forest Classifier

Code Snippet

```python
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)
rf_model.fit(X_train, y_train)

scores = cross_val_score(rf_model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

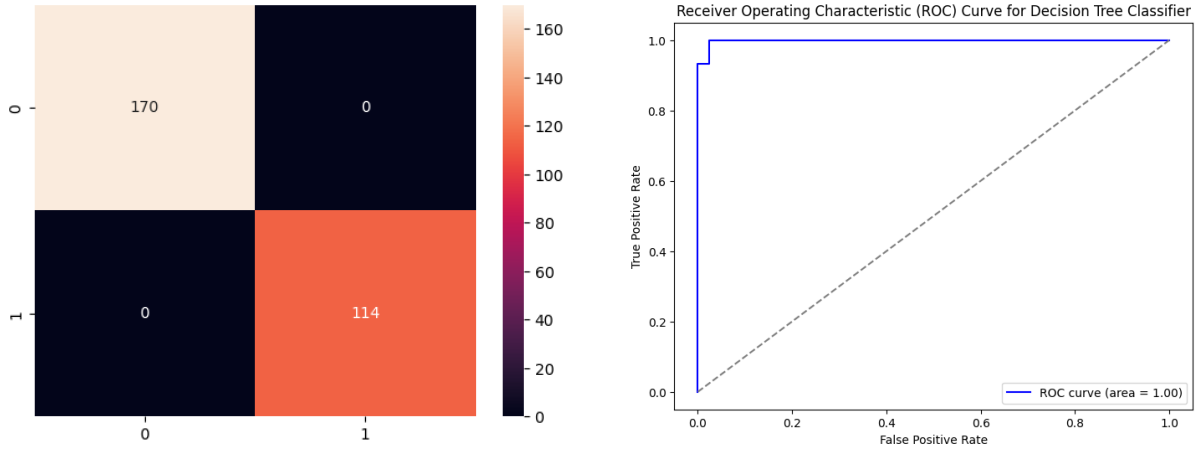
Figure 2: Random Forest Evaluation

AdaBoost Classifier

Code Snippet

```
from sklearn.ensemble import AdaBoostClassifier

ada_model = AdaBoostClassifier(
    n_estimators=100,
    learning_rate=1.0,
    random_state=42 )
ada_model.fit(X_train, y_train)

scores = cross_val_score(ada_model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

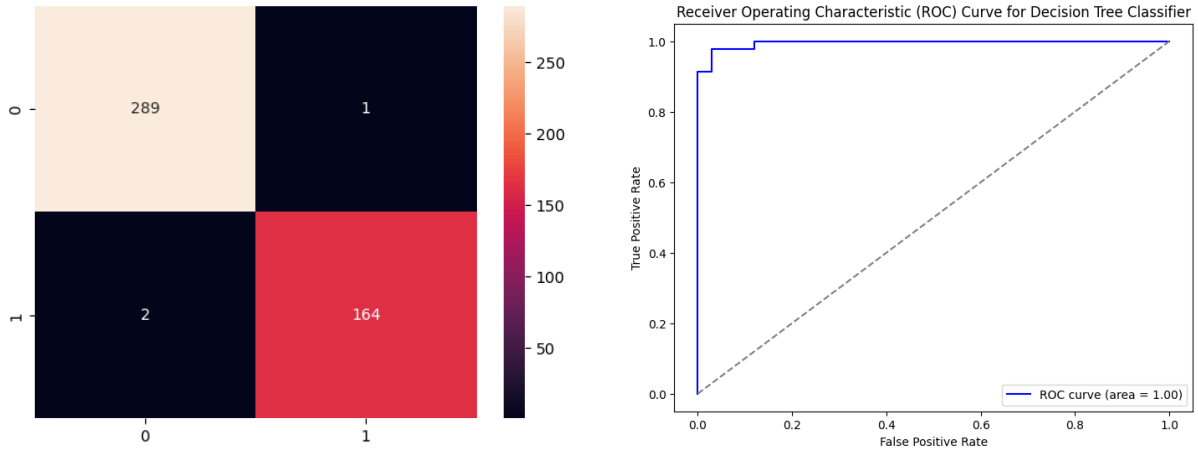
Figure 3: AdaBoost Evaluation

Gradient Boost Classifier

Code Snippet

```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
gb_model.fit(X_train, y_train)

scores = cross_val_score(gb_model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

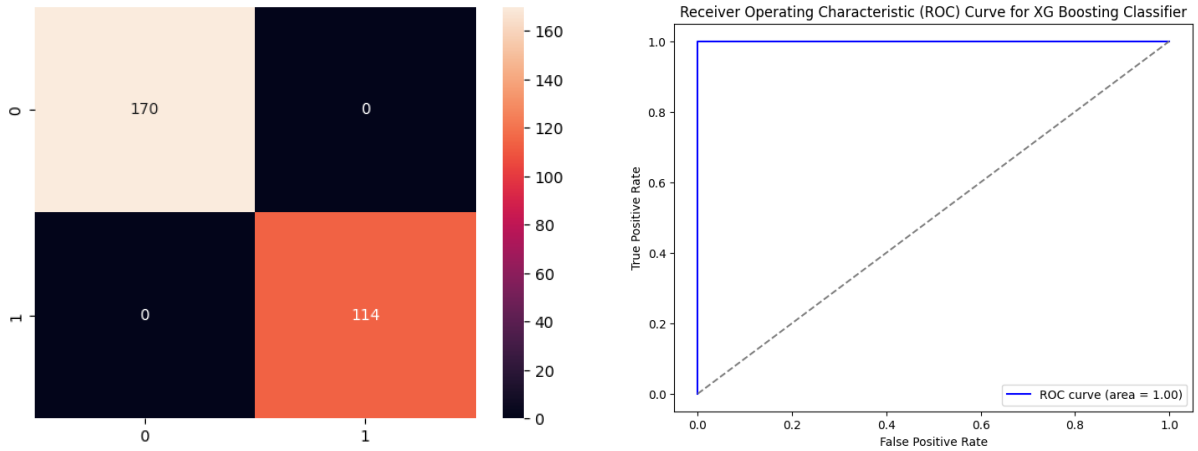
Figure 4: Gradient Boosting Evaluation

XGBoost Classifier

Code Snippet

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)
xgb_model.fit(X_train, y_train)

scores = cross_val_score(xgb_model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

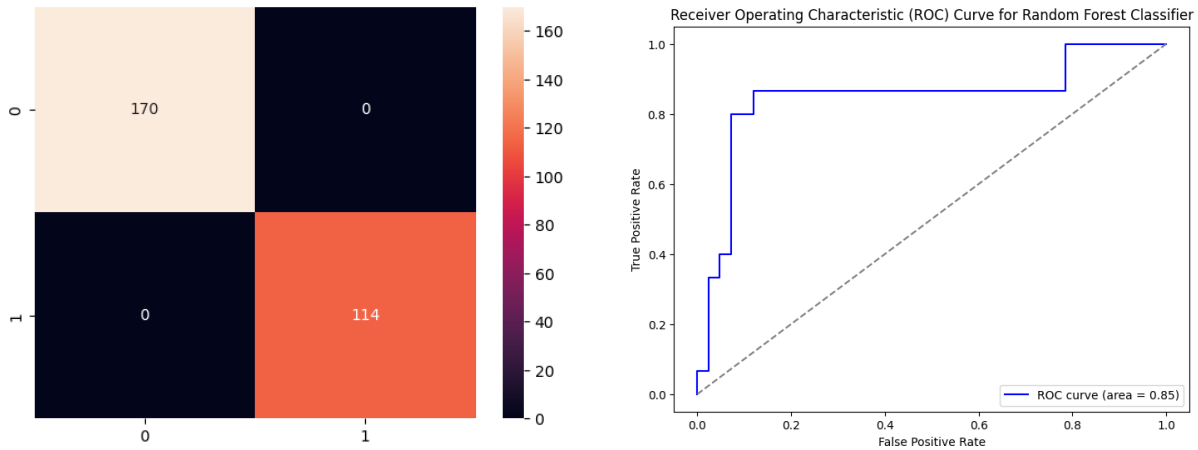
Figure 5: XGBoost Evaluation

Stacked Ensemble Classifier

Code Snippet

```python
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

base_learners = [
    ("svm", SVC(probability=True, random_state=42)),
    ("nb", GaussianNB()),
    ("dt", DecisionTreeClassifier(random_state=42))
]

final_estimator = LogisticRegression(random_state=42)

model = StackingClassifier(
    estimators=base_learners,
    final_estimator=final_estimator,
    cv=5,
    n_jobs=-1
)

scores = cross_val_score(model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

Plots



(a) Confusion Matrix (Testing)

(b) ROC Curve (Testing)

Figure 6: Stacked Ensemble Evaluation

Performance Metrics of Testing :-

Metric	Accuracy	Precision	F1 Score	Recall
Decision Tree	0.94	0.95	0.93	0.91
Random Forest	1.0	1.0	1.0	1.0
AdaBoost Classifier	0.98	1.0	0.96	0.93
Gradient Boost	0.94	0.92	0.89	0.86
Extreme Gradient Boosting	0.98	0.93	0.96	1.0
Stacked Ensemble Classifier	0.89	0.8	0.8	0.8

Table : Performance Metrics of all Models

Hyperparameter Trials :-

Decision Tree Model

criterion	max depth	Accuracy	F1 Score
Entropy	20	0.9469	0.933

Table 1: Decision Tree – Hyperparameter Tuning

AdaBoost Model

n estimators	learning rate	Accuracy	F1 Score
200	0.5	0.9824	0.9655

Table 2: AdaBoost – Hyperparameter Tuning

Gradient Boosting Model

n estimators	learning rate	max depth	Accuracy	F1 Score
200	0.05	2	0.96460	0.95454

Table 3: Gradient Boosting – Hyperparameter Tuning

XGBoost Model

n estimators	learning rate	max depth	gamma	Accuracy	F1 Score
200	0.2	3	0	0.98	0.96

Table 4: XGBoost – Hyperparameter Tuning

Random Forest Model

n estimators	max depth	criterion	Accuracy	F1 Score
100	30	Entropy	0.9824	0.9655

Table 5: Random Forest – Hyperparameter Tuning

Stacked Ensemble Model

Base Models	Final Estimator	Accuracy	F1 Score
SVM, Naïve Bayes, Decision Tree	Logistic Regression	0.89	0.8
SVM, Naïve Bayes, Decision Tree	Random Forest	0.84	0.8
SVM, Decision Tree, KNN	Logistic Regression	0.75	0.36

Table 6: Stacked Ensemble – Hyperparameter Tuning

5-Fold Cross-Validation Results

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average Accuracy
Decision Tree	0.956140	0.947368	0.885965	0.929825	0.955752	0.935010
AdaBoost	0.964912	0.973684	0.956140	0.973684	0.964602	0.966605
Gradient Boosting	0.964912	0.973684	0.956140	0.973684	0.964602	0.966605
XGBoost	0.973684	0.982456	0.938596	0.982456	0.973451	0.970129
Random Forest	0.964912	0.973684	0.947368	0.956140	0.946903	0.957802
Stacked Model	0.938596	0.921053	0.912281	0.964912	0.938053	0.934979

Table 7: 5-Fold Cross Validation Results for All Models

Observations and Conclusion :-

This experiment implemented and evaluated six ensemble classification models. Performance was assessed using standard metrics, 5-fold cross-validation, and visual analysis.

Best Classifier by Average Accuracy

The **XGBoost classifier** achieved the highest average cross-validation accuracy of **97.01%**, identifying it as the most consistent and reliable model across different data subsets.

Effect of Hyperparameters

Hyperparameter tuning was critical for performance:

- **Boosting Models (AdaBoost, GB, XGBoost):** Increasing `n_estimators` with a conservative `learning_rate` significantly boosted accuracy and F1-scores.
- **Random Forest:** A larger `max_depth` and `criterion='entropy'` improved model flexibility and performance.
- **Stacked Ensemble:** Performance was highly sensitive to the choice of base learners and the final estimator.

Generalization Across CV Folds

XGBoost and AdaBoost demonstrated the best generalization, with high accuracy scores that were consistently strong across all folds (low variance). The Stacked Ensemble showed higher variability between folds, indicating less consistent generalization.

ROC Curves and Confusion Matrices

- **ROC Curves:** The curves for top models (Random Forest, XGBoost, AdaBoost) were closest to the top-left corner, indicating excellent classification performance.
- **Confusion Matrices:** Visually confirmed the quantitative metrics, with Random Forest showing a perfect diagonal on the test set and other models showing proportionally more errors.

Overall Conclusion

The **XGBoost classifier** is concluded to be the best model for this dataset, offering an optimal balance of high accuracy and robust generalization. The results highlight the importance of cross-validation over a single train-test split for reliable model assessment and the significant gains achievable through hyperparameter tuning.

Learning Outcomes

- I have learnt various models of Classification such as
    1. Decision Tree classifier
    2. AdaBoost Classifier
    3. Gradient Boosting Classifier
    4. Extreme Boosting Classifier
    5. Random Forest Classifier
    6. Stacked Ensemble Classifier
- I am able to Perform EDA,Hyperparameter Tuning,Model,Cross Validation for all models.
- I am able to evaluate the testing using Confusion Matrix,ROC Curve,Accuracy,Precision.